



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 01

NOMBRE COMPLETO: Loza Regino Miguel Ángel

N° de Cuenta: 319520011

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 07

SEMESTRE 2027-1

FECHA DE ENTREGA LÍMITE: 02/09/2026

CALIFICACIÓN: _____

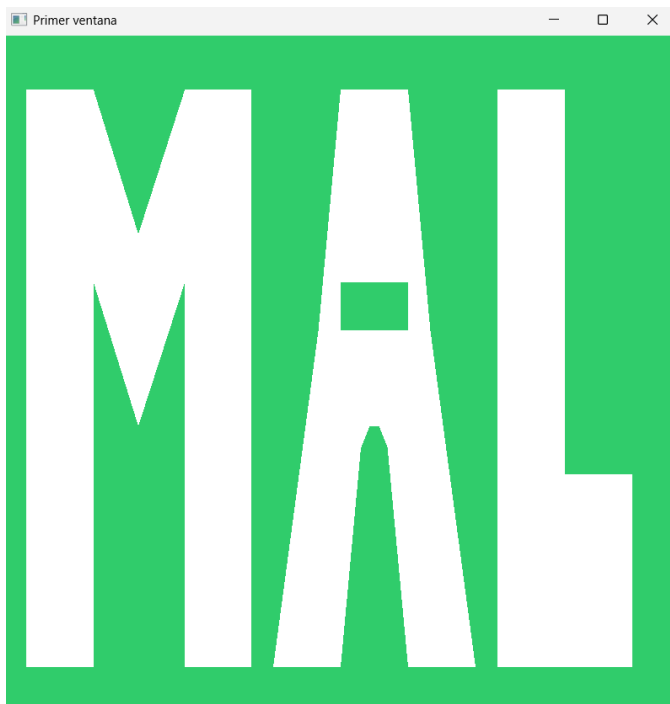
Reporte de práctica 1: Introducción a OpenGL

Se realizó el siguiente ejercicio:

1. Ventana cambia el color de fondo de forma aleatoria tomando rango de colores RGB y con una periodicidad de 2 segundos. (Verificar que al ejecutar el programa varias veces el orden de los colores si lo vean aleatorio y no siempre los mismos).
2. 3 letras iniciales de sus nombres creadas a partir de triángulos, todas las letras son del mismo color.

Los dos ejercicios se muestran de forma simultánea y están en el mismo main.

Ejecución del ejercicio:







Se adjunta de igual manera una liga a un video donde se muestra el funcionamiento:

<https://drive.google.com/file/d/1kWta-tlC27zsFTNIT8pD6YneUijXCyRx/view?usp=sharing>

Bloques de código:

Color de fondo aleatorio

Utilicé las siguientes bibliotecas

```
#include <stdlib.h>
#include <time.h>
```

Usé `stdlib.h` para poder usar las funciones `rand()` y `srand()`.

`rand()` es quien genera los números aleatorios para armar el color RGB, y `srand()` es la que fija la semilla desde donde `rand()` empieza a generar esa secuencia.

Usé `time.h` para la función `time(NULL)`, que me da los segundos actuales desde 1970.

Usé `time(NULL)` ya que cambia en cada ejecución. Eso hace que la semilla sea distinta cada vez y `rand()` arranca en un punto distinto de su secuencia.

```
float bgR = 0.0f, bgG = 0.0f, bgB = 0.0f;
double ultimoCambio = 0.0;
const double INTERVALO = 2.0;
```

float bgR = 0.0f, bgG = 0.0f, bgB = 0.0f; Son las variables del RGB del fondo. Son globales porque se necesitan leer y modificar dentro del if que genera el color nuevo y en `glClearColor()` que lo aplica.

Empiezan en 0.0f, lo que hace que al iniciar siempre sea negro antes del primer `rand()`.

double ultimoCambio = 0.0; Guarda el timestamp del último momento en que se generó un color nuevo.

const double INTERVALO = 2.0; Es el intervalo de cambio

```
srand((unsigned int)time(NULL));
```

Toma la hora del sistema en segundos y la casteamos con **unsigned int** para usarla de semilla con **srand**.

if(tiempoActual-ultimoCambio>=INTERVALO)

Aquí se compara tiempo de ahora menos tiempo del último cambio de color. Si esa diferencia ya llegó a 2, entra al bloque y genera color nuevo.

Se genera un número aleatorio para cada variable RGB y se normalizan a 0.0–1.0 dividiendo entre `RAND_MAX`.

Se actualiza `ultimoCambio = tiempoActual`, para mantener el intervalo de 2s.

```
//Loop mientras no se cierra la ventana
while (!glfwWindowShouldClose(mainWindow))
{
    //Recibir eventos del usuario
    glfwPollEvents();

    double tiempoActual = glfwGetTime();

    if (tiempoActual - ultimoCambio >= INTERVALO)
    {
        bgR = (float)rand() / (float)RAND_MAX;
        bgG = (float)rand() / (float)RAND_MAX;
        bgB = (float)rand() / (float)RAND_MAX;
        ultimoCambio = tiempoActual;
    }

    glClearColor(bgR, bgG, bgB, 1.0f);

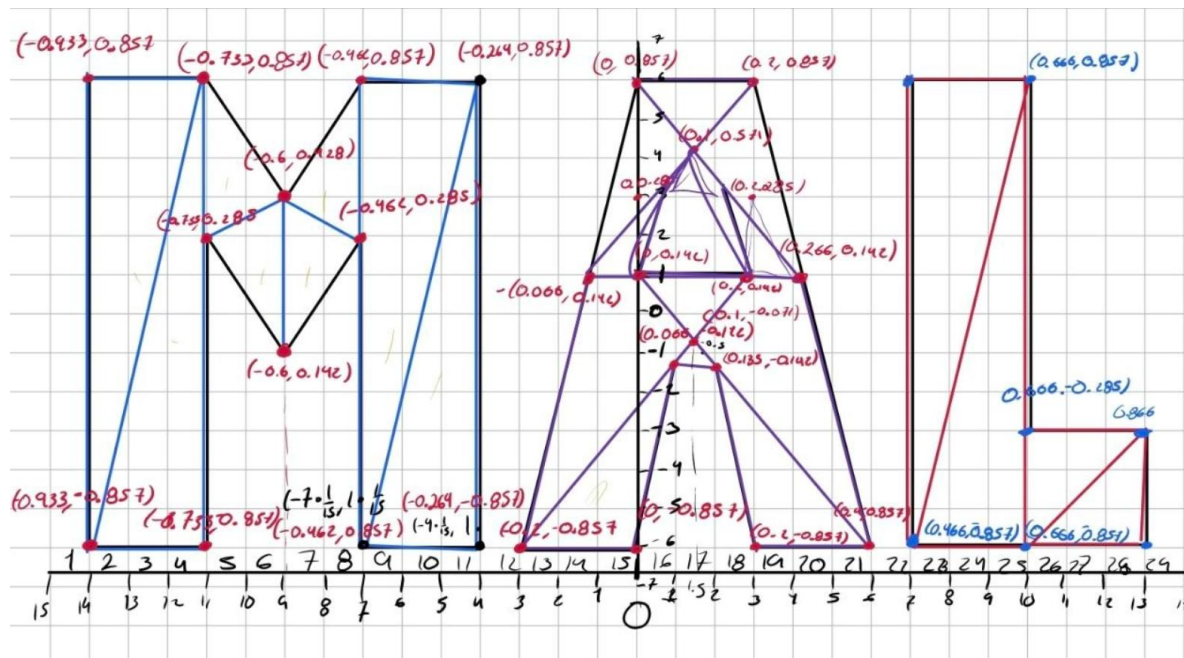
    //Limpiar la ventana
    glClear(GL_COLOR_BUFFER_BIT);
```

Triángulos que forman las letras M, A, L. (Miguel Ángel Loza)

```
static const char* fShader = "          \n\
#version 330                      \n\
out vec4 color;                  \n\
void main()                      \n\
{                                \n\
    color = vec4(1.0f,1.0f,1.0f,1.0f); \n\
}";
```

Definí el color a usar en blanco con los valores 1.0 para todas las variables RGB.

Fue preciso usar 23 triángulos con un total de 69 vertices para la formación de las letras mi nombre.



```

void CrearTriangulo()
{
    GLfloat vertices[] = {
        -0.264f, -0.857f, 0.0f,
        -0.462f, -0.857f, 0.0f,
        -0.264f, 0.857f, 0.0f,

        -0.462f, 0.857f, 0.0f,
        -0.462f, -0.857f, 0.0f,
        -0.264f, 0.857f, 0.0f,

        -0.462f, 0.857f, 0.0f,
        -0.462f, 0.285f, 0.0f,
        -0.6f, 0.428f, 0.0f,

        -0.6f, -0.142f, 0.0f,
        -0.462f, 0.285f, 0.0f,
        -0.6f, 0.428f, 0.0f,

        -0.6f, -0.142f, 0.0f,
        -0.733f, 0.285f, 0.0f,
        -0.6f, 0.428f, 0.0f,

        -0.733f, 0.857f, 0.0f,
        -0.733f, 0.285f, 0.0f,
        -0.6f, 0.428f, 0.0f,

        -0.733f, 0.857f, 0.0f,
        -0.733f, -0.857f, 0.0f,
        -0.933f, -0.857f, 0.0f,

        -0.733f, 0.857f, 0.0f,
        -0.933f, 0.857f, 0.0f,
        -0.933f, -0.857f, 0.0f,

        -0.2f, -0.857f, 0.0f,
        -0.0f, -0.857f, 0.0f,
        0.066f, -0.142f, 0.0f,

        0.1f, -0.071f, 0.0f,
        0.133f, -0.142f, 0.0f,
        0.066f, -0.142f, 0.0f,

        0.2f, -0.857f, 0.0f,
        0.133f, -0.142f, 0.0f,
        0.4f, -0.857f, 0.0f,

        0.0f, 0.142f, 0.0f,
        0.266f, 0.142f, 0.0f,
        0.4f, -0.857f, 0.0f,

        0.2f, 0.142f, 0.0f,
        -0.066f, 0.142f, 0.0f,
        -0.2f, -0.857f, 0.0f,

        0.0f, 0.857f, 0.0f,
        -0.066f, 0.142f, 0.0f,
        0.1f, 0.571f, 0.0f,

        0.2f, 0.857f, 0.0f,
        0.266f, 0.142f, 0.0f,
        0.1f, 0.571f, 0.0f,

        -0.03f, 0.285f, 0.0f,
        0.23f, 0.285f, 0.0f,
        0.1f, 0.571f, 0.0f,

        0.2f, 0.571f, 0.0f,
        0.266f, 0.142f, 0.0f,
        0.2f, 0.142f, 0.0f,

        0.0f, 0.571f, 0.0f,
        -0.066f, 0.142f, 0.0f,
        0.0f, 0.142f, 0.0f,

        0.0f, 0.857f, 0.0f,
        0.1f, 0.571f, 0.0f,
        0.2f, 0.857f, 0.0f,
    }
}

```

```

0.466f, -0.857f, 0.0f,
0.666f, -0.857f, 0.0f,
0.666f, 0.857f, 0.0f,

0.466f, -0.857f, 0.0f,
0.466f, 0.857f, 0.0f,
0.666f, 0.857f, 0.0f,

0.866f, -0.285f, 0.0f,
0.666f, -0.285f, 0.0f,
0.666f, -0.857f, 0.0f,

0.866f, -0.285f, 0.0f,
0.866f, -0.857f, 0.0f,
0.666f, -0.857f, 0.0f

```

Definí el arreglo vertices en la función CrearTriangulo(). Donde se encuentran las coordenadas de los 69 vértices usados.

```
glBindVertexArray(VAO);  
glDrawArrays(GL_TRIANGLES, 0, 69);  
glBindVertexArray(0);
```

Aquí delimite la cantidad de vértices de los triángulos.

Errores y solución de estos:

Durante la ejecución en el proceso de introducir los vértices, llego un momento donde lo pude volver a ejecutar el programa ya que Windows defender con una función llamada control inteligente de aplicaciones, al desactivarla ya pude seguir con el trabajo.

Así mismo en un inicio tuve problemas con la forma en la que estaba escalado las letras. Solo volví a realizar el escalado y no tuve más problemas.

Conclusión:

Los ejercicios del reporte: En general el ejercicio no me resulto difícil. Aunque si fue tedioso tener que escalar las letras y anotar todas las coordenadas de los vértices, ya que era muy fácil confundirse. Por otro la lógica fue sencilla de programar.

Comentarios generales: Considero que en la practica la explicación fue muy rápida, aunque entiendo que de ser más lenta no habría dejado nada de tiempo para realizar el ejercicio de clase, sin embargo, considero que podríamos tratar de ir un poco más despacio. También considero que deberíamos usar formato IEEE en vez de formato APA, no solemos usar a pesar de ser un formato enfocado en nuestra rama.

Conclusión:

El ejercicio del color aleatorio me dejó clara la importancia de usar una semilla basada en tiempo real para que rand() no genere siempre la misma secuencia, y el de las letras con triángulos me ayudó a practicar la ubicación de vértices en el plano cartesiano de -1 a 1, aunque fue bastante tedioso calcular y ajustar cada coordenada a mano. En general, la lógica detrás de ambos ejercicios no fue complicada; lo que más trabajo me costó fue la parte manual de escalar y posicionar las letras correctamente.

Bibliografía

GLFW. (s. f.). *Getting started: A simple GLFW application*. GLFW Documentation. <https://www.glfw.org/docs/latest/quick.html>

GLFW. (s. f.). *GLFW: A multi-platform library for OpenGL, OpenGL ES, Vulkan, window and input* [Repositorio de código]. GitHub. <https://github.com/glfw/glfw>

Gerdelan, A. (2016). *Hello triangle: Anton's OpenGL 4 tutorials*. <https://antongerdelan.net/opengl/hellotriangle.html>